

Question - 1:

1. Naive, Single-stage image size - 2.28 GB
2. Multi Stage image size - 718 MB
Percentage reduction = $((2.28 - 0.718)/2.28)*100 = 68.5 \%$
3. The multi stage image is smaller because we use the full python:3.12 only for installing dependencies and then switch to the lighter python:3.12-slim image for running the API. For spam-API, the larger builder environment and its build/install files are left behind, while only the required dependencies, `app.py`, and `model.joblib` are copied into the final image.

Question - 2:

3. **Cache timing evidence:** Two identical requests were sent to the API. The first request (cache MISS) took **0.070948 seconds**, while the second request (cache HIT) took **0.002123 seconds**, giving a measured speedup of **33.42x**.
4. Docker Compose runs and connects multiple containers as one application stack, while Kubernetes provides cluster-level orchestration with scaling, service management, and self-healing.

Question - 3:

2. The cluster has 2 nodes, with 2 allocatable CPUs on each node:
Node 1: 2 CPUs
Node 2: 2 CPUs - Total allocatable CPU = 2 + 2 = 4 CPUs.
Each Indexed Job Pod requested 1 CPU, and the cluster had 4 allocatable CPUs in total, so **parallelism: 4** allowed up to four Pods to run concurrently. The execution timestamps showed overlapping Pod execution, confirming that the shards were processed in parallel rather than sequentially. **parallelism: 4** is the maximum concurrency, so fewer Pods may run at a given instant depending on scheduling.

pod execution times

INDEX	POD	START	FINISH
0	shard-validator-job-0-np647	2026-09-18T05:52:16Z	2026-09-18T05:53:16Z
1	shard-validator-job-1-k6czt	2026-09-18T05:52:16Z	2026-09-18T05:53:16Z
2	shard-validator-job-2-g42cw	2026-09-18T05:53:18Z	2026-09-18T05:54:18Z
3	shard-validator-job-3-zr4dl	2026-09-18T05:53:19Z	2026-09-18T05:54:19Z
4	shard-validator-job-4-88296	2026-09-18T05:54:21Z	2026-09-18T05:55:21Z
5	shard-validator-job-5-lvjsq	2026-09-18T05:55:23Z	2026-09-18T05:56:23Z
6	shard-validator-job-6-kzwcc	2026-09-18T05:54:21Z	2026-09-18T05:55:21Z
7	shard-validator-job-7-lhfzm	2026-09-18T05:55:24Z	2026-09-18T05:56:24Z

3. I collected results by reading Pod logs through the Kubernetes API instead of using a shared volume. Since each Pod produced only a small result and the workload was short-lived, this approach was simpler and avoided additional shared-storage setup.

Results collected through kubernetes API:

completion_index	shard	total_rows	invalid_rows	valid_rows	pod_name	node_name
0	shard-0.csv	100	3	97	shard-validator-job-0-np647	q3-cluster-m02
1	shard-1.csv	100	3	97	shard-validator-job-1-k6czt	q3-cluster
2	shard-2.csv	100	3	97	shard-validator-job-2-g42cw	q3-cluster
3	shard-3.csv	100	3	97	shard-validator-job-3-zr4dl	q3-cluster-m02
4	shard-4.csv	100	3	97	shard-validator-job-4-88296	q3-cluster
5	shard-5.csv	100	3	97	shard-validator-job-5-lvjsq	q3-cluster
6	shard-6.csv	100	3	97	shard-validator-job-6-kzwcc	q3-cluster-m02
7	shard-7.csv	100	3	97	shard-validator-job-7-lhfzm	q3-cluster-m02

Summary:
Pods found: 8
Results collected: 8
Total rows processed: 800
Total invalid rows: 24
Total valid rows: 776

4. With 3 nodes and 2 allocatable CPUs per node, the cluster would have 6 allocatable CPUs in total. Since each Job Pod requests 500m CPU, up to 12 Pods could theoretically fit based purely on CPU requests, but the assignment's stated 2-CPU-per-node capacity and the intended one-worker-per-CPU design would support a **parallelism** of 6. With 8 completions, six shards

could run concurrently and the remaining two would run afterward.

Question - 4:

2. Before Deletion

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
spam-api-6d9b749fc7-v87vj	1/1	Running	0	5m21s	10.244.1.48	q3-cluster-m02	<none>	<none>
spam-api-6d9b749fc7-w7g57	1/1	Running	0	63s	10.244.0.46	q3-cluster	<none>	<none>

After Deletion

NAME	READY	STATUS	RESTARTS	AGE
spam-api-6d9b749fc7-6hstj	1/1	Running	0	2m58s
spam-api-6d9b749fc7-v87vj	1/1	Running	0	2m58s

Self-healing: Initially, the Deployment had two running replicas. After manually deleting Pod `spam-api-6d9b749fc7-6hstj`, Kubernetes automatically created a new Pod `spam-api-6d9b749fc7-w7g57`, restoring the desired replica count of two.

The **Deployment controller**, through the ReplicaSet it manages, maintains the desired number of Pods. When a Pod is deleted, the ReplicaSet detects that the actual number of replicas is below the desired count and creates a replacement Pod.

3. A visible version change was introduced in the `/healthz` response, changing it to `{"status":"ok","version":"v2"}`. The updated application was built as `spam-api-k8s:v2`, the Deployment was updated to use this image, and `kubectl rollout status` confirmed that the rollout completed successfully; `kubectl rollout history` showed revisions 1 and 2. If this application were deployed as a **Job**, it would be intended for a finite task that runs to completion rather than a continuously running service. Updating the Job would not provide the same rolling-update mechanism used by a Deployment, which manages long-running replicas and gradually replaces old Pods with new ones. A Job would instead run its specified workload and terminate after completion.

```
/kubectl rollout status deployment/spam-api
✓ 0.9s Python
deployment "spam-api" successfully rolled out

t pods -l app=spam-api \
  --columns="NAME:.metadata.name,IMAGE:.spec.containers[0].image,READY:.status.containerStatuses[0].ready,STATUS:.status.phase"
✓ 0.8s Python
NAME          IMAGE          READY  STATUS
spam-api-8475c587cd-d9xlt spam-api-k8s:v2 true    Running
spam-api-8475c587cd-l8hbp spam-api-k8s:v2 true    Running

/kubectl rollout history deployment/spam-api
✓ 0.8s Python
deployment.apps/spam-api
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

4. Deployment vs Kubernetes Job

A Kubernetes **Deployment** is designed for long-running applications such as our spam-detection API. It maintains the desired number of replicas, replaces failed Pods, and supports rolling updates when the application image changes.

A Kubernetes **Job** is designed for finite batch workloads. In Q3, the Indexed Job processed the 8 CSV shards and completed each task, whereas the Q4 Deployment keeps the API replicas running continuously and maintains the desired state.